

name: java-spring-conventions

description: plantynet Spring Boot 프로젝트의 코딩 컨벤션과 아키텍처 패턴을 적용. Java 코드 생성, 클래스/메서드 추가, 레이어 구조 설계, 리팩토링 작업 시 반드시 사용. Allman brace style, 탭 들여쓰기, MyBatis(JPA 미사용), 명시적 Constructor Injection, Lombok 제한 규칙 준수가 필요할 때 항상 이 스킬을 활성화. Spring Boot 프로젝트에서 새 파일 생성, 기존 코드 수정, 코드 리뷰 요청 시에도 적용.

Java Spring Boot 코딩 컨벤션 (plantynet / gardenapi)

이 스킬은 Spring Boot 프로젝트에서 일관된 코딩 스타일과 아키텍처 패턴을 유지하기 위한 가이드라인이다. 새 파일을 생성하거나 기존 코드를 수정할 때 반드시 아래 규칙을 따른다.

프로젝트: gardenapi (`com.plantynet.gardenapi`) — Spring Boot 3.5.14, Java 17, MyBatis 3.0.5

변들 파일:

- `references/intellij-code-style.xml` — IntelliJ Code Style 설정 (`.idea/codeStyles/Project.xml` 에 복사)
- `references/lombok.config` — Lombok 허용/금지 규칙 (`src/main/java/lombok.config`)
- `references/lombok-conventions.md` — 클래스 유형별 Lombok 어노테이션 사용 기준 상세
- `references/controller.md` — 컨트롤러 작성 규칙 상세
- `references/service.md` — 서비스 작성 규칙 상세
- `references/dao.md` — DAO / MyBatis XML 작성 규칙 상세
- `references/entity.md` — Entity 작성 규칙 상세
- `references/dto.md` — DTO 작성 규칙 상세

1. 브레이스 스타일 — Allman Style (필수)

여는 중괄호 `{` 를 항상 새로운 줄에 위치시킨다. K&R이나 1TBS 스타일은 사용하지 않는다.

```
// 올바른 예
public class UserService
{
    public User findById(String userId)
    {
        if (userId == null || userId.isBlank())
        {
            throw new IllegalArgumentException("userId must not be blank");
        }
        return userDao.selectUser(userId);
    }
}
```

`else`, `catch`, `finally` 도 닫는 `}` 다음 줄에 위치한다.

```
if (condition)
{
    doA();
}
else
{
    doB();
}

try
{
    process();
}
catch (Exception e)
{
    log.error("오류", e);
}
finally
{
    cleanup();
}
```

2. 들여쓰기 — Tab (필수)

공백(space)이 아닌 **탭(tab)** 문자로 들여쓰기한다. 1 레벨 = 1 탭.

3. 패키지 레이어 구조

com.plantynet.gardenapi/	
├── GardenapiApplication.java	
├── ctl/	— REST 컨트롤러 (@RestController)
│ ├── BaseController.java	— 공통 응답/인증 기반 클래스
│ └── v1/	— 실 서비스 컨트롤러 (버전별 하위 패키지)
├── service/	— 서비스 클래스 (인터페이스 없이 직접 @Service)
├── dao/	— MyBatis 매퍼 인터페이스
├── entity/	— DB 결과 매핑 VO
├── dto/	— API 입출력 전용 DTO
│ ├── request/	— 요청 DTO (*Req suffix)
│ └── response/	— 응답 DTO (*Resp suffix)
├── common/	
│ ├── enums/	— ApiMessage, UserRole, SocialProvider, UploadTarget 등
│ └── util/	— 순수 유틸리티 (TSID, XSS, CodeUtil 등)
│ ├── network/	— RestClientUtil
│ └── xss/	— JsonHtmlXssSerializer, JsonHtmlXssDeserializer
├── config/	— Spring 설정 (@Configuration)
│ └── azure/	— AzureBlobConfig, AzureBlobProperties
└── support/	— 공통 기반 기술 (비즈니스 로직 없음)
├── ClonedTaskDecorator.java	— MDC + SecurityContext 비동기 전파
├── aop/	— LoggingAspect
├── exception/	— DomainException 하위 커스텀 예외
├── filter/	— MethodFilter, JsonHiddenHttpMethodFilter
├── handler/	— ControllerExceptionHandler (전역 예외 처리)
├── interceptor/	— MePathVariableInterceptor
└── jwt/	— JwtTokenProvider, CustomUserDetails, JwtAuthenticationFilter,
JwtAuthenticationEntryPoint	
├── mapper/	— MapStruct 매퍼 인터페이스
└── redis/	— KeyMaker (인터페이스), KeyMakerResolver

4. 네이밍 컨벤션

대상	규칙	예시
클래스	PascalCase	UserService , AuthController
컨트롤러	도메인 + Controller	UserController , AuthController
서비스	도메인 + Service	UserService , RedisService
DAO	도메인 + Dao	UserDao , CodeDao
Entity	도메인명	User , FamilyMember , AuthProvider
Request DTO	도메인 + Req	FileDownloadReq , FileUploadReq
Response DTO	도메인 + Resp	FileUploadResp , FileDownloadResp , UserResp
공통 DTO	역할 설명적 이름	ApiResponse , DeviceInfo
Enum	PascalCase	UserRole , ApiMessage , SocialProvider
메서드	camelCase, 동사 시작	selectAuthInfo , updateLastLoginAt
상수	UPPER_SNAKE_CASE	ALLOWED_METHODS
필드	camelCase	userId , familyId , accessToken
MyBatis XML	도메인명.xml	User.xml , Code.xml

5. 의존성 주입

Controller / Service: @RequiredArgsConstructor 허용

```
@Slf4j
@Service
@RequiredArgsConstructor
public class UserService
{
    private final UserDao userDao;
    private final UserMapper userMapper;
}
```

Config / Support: 명시적 생성자 직접 작성

```
@Configuration
public class AppConfig
{
    private final CodeService codeService;

    public AppConfig(CodeService codeService)
    {
        this.codeService = codeService;
    }
}
```

금지: @Autowired 필드 주입

6. Lombok 사용 규칙

허용된 어노테이션

어노테이션	사용 위치
@Getter	Entity / DTO
@Setter	Request DTO (Jackson 주입용)
@ToString	Entity / DTO (선택)
@Builder	Entity / Response DTO
@NoArgsConstructor(access = PROTECTED)	Entity / Response DTO (@Builder 와 함께)
@AllArgsConstructor(access = PRIVATE)	Entity / Response DTO (패키지별 lombok.config로 허용)
@RequiredArgsConstructor	Controller / Service / Enum
@Slf4j	Controller / Service / Filter / AOP

클래스 유형별 상세 기준: [references/lombok-conventions.md](#) 참조

금지된 어노테이션 (lombok.config — flagUsage=error)

@Data, @Value (Lombok), val, var, @NonNull, @Cleanup, @SneakyThrows, @Synchronized, @experimental.*

@AllArgsConstructor 는 전역 기본값이 error이나, entity/ 및 dto/response/ 패키지의 lombok.config에서 allow로 override된다.

@Setter 는 entity/ 및 dto/response/ 패키지에서 error로 금지된다.

7. 트랜잭션 관리 — AOP 기반 (직접 선언 금지)

`@Transactional` 을 서비스 메서드에 직접 붙이지 않는다. `TransactionConfig` (AOP)가 자동 적용한다.

- `select*`, `get*` → `PROPAGATION_SUPPORTS` (읽기 전용)
- 나머지 메서드 → `PROPAGATION_REQUIRED`
- Pointcut: `execution(* com.plantynet.*.service.*Service.*(..))`

8. 예외 처리

커스텀 예외는 `DomainException` (추상)을 상속한다.

```
// 기본형 — ApiMessage만 전달
public class UserNotFound extends DomainException
{
    public UserNotFound()
    {
        super(ApiMessage.USER_NOT_FOUND);
    }
}

// 오버로드형 — 호출 측에서 ApiMessage 선택 (Forbidden 등)
public class Forbidden extends DomainException
{
    public Forbidden()
    {
        super(ApiMessage.FORBIDDEN);
    }

    public Forbidden(ApiMessage apiMessage)
    {
        super(apiMessage);
    }
}
```

비즈니스 로직에서 직접 던지기:

```
User user = userDao.selectAuthInfo(provider, providerUid)
    .orElseThrow(() -> new UserNotFound());
```

`ControllerExceptionHandler` (`support/handler/`)가 `DomainException` → 가변 상태코드, validation 예외 → 400 처리.

9. 응답 형식 — ApiResponse

모든 API 응답은 `ApiResponse<?>` 래퍼를 사용한다.

```
{ "statusCode": 200, "status": 200, "message": "성공", "data": {...} }  
{ "statusCode": 404, "status": 40401, "message": "사용자를 찾을 수 없음", "error": "USER_NOT_FOUND" }
```

- `ApiResponse.success()` / `success(data)` — 200
- `ApiResponse.created()` / `created(data)` — 201
- `ApiResponse.of(ApiMessage)` — 에러 응답 (`error` = enum 이름, `message` = 설명)

`ApiMessage` 구조:

```
USER_NOT_FOUND(404, 40401, "사용자를 찾을 수 없음")  
// httpCode, status(상세코드), description
```

10. 로깅 패턴

`@Slf4j` 사용. topic은 선택이며, 도메인/기능을 나타내는 영어 대문자로 지정한다.

```
@Slf4j(topic = "USER") // topic 있는 경우  
@Slf4j // topic 없는 경우도 허용
```

LoggingAspect 자동 적용: `execution(* com.plantynet.*.service.*Service.*(..))` → 서비스 메서드 입출력 + 실행시간 DEBUG 자동 로깅.

11. 보안 — 요청 처리 흐름

HTTP 요청
→ `MethodFilter` (GET, POST만 통과 / 그 외 405)
→ `JsonHiddenHttpMethodFilter` (X-HTTP-Method-Override: PUT/PATCH/DELETE 지원)
→ `JwtAuthenticationFilter` (Bearer 토큰 → `SecurityContext`)
→ Spring Security (인가)
→ `MePathVariableInterceptor` (`{familyId}/{userId}=me` → JWT 클레임 대치)
→ Controller

HTTP 메서드 정책: 실제 HTTP는 GET/POST만 허용. PUT/DELETE는 POST 요청에 `X-HTTP-Method-Override` 헤더로 전달한다. `@DeleteMapping`, `@PutMapping` 은 컨트롤러에서 사용 가능 (필터가 변환).

JWT 클레임: `userId` (subject), `familyId`, `role` (예: `ROLE_RS002`).

SecurityContext 접근: `BaseController.customUserDetails()` 로 `CustomUserDetails` 획득.

```
customUserDetails().getUserId()  
customUserDetails().getFamilyId()  
customUserDetails().getAccessToken()
```

`@AuthenticationPrincipal CustomUserDetails userDetails` 를 컨트롤러 메서드 파라미터로 받는 방식도 동일하게 동작한다.

프로필별 Security 정책:

- local : /v1/**, /v2/** permitAll
- 그 외: /v1/auth/sign-in, /v1/auth/token/refresh 만 permitAll

me 경로 변수:

- {familyId} 또는 {userId} 자리에 "me" 를 전달하면 MePathVariableInterceptor 가 JWT 클레임 값으로 자동 대치
- 일반 역할(RS001~RS004): "me" 사용 가능
- 관리자 역할(AD001, AD002): "me" 사용 불가 (명시적 ID 필수)

12. 비동기 처리

@Async("gardenTask") 메서드는 반드시 별도 클래스에 선언한다 (자기 호출 프록시 우회 방지).
ClonedTaskDecorator 가 MDC + SecurityContext를 비동기 스레드로 전파한다.

13. ID 생성 — TSID

```
long id = TsidCreator.getTsid().toLong();  
String idStr = TsidCreator.getTsid().toString(); // 13자리 Crockford Base32
```

14. XSS 방어

JsonHtmlXssSerializer / JsonHtmlXssDeserializer Jackson 모듈 + Lucy XSS Filter가 전역 적용. 별도 처리 불필요.

15. Import 순서

1. java.*
2. javax.* / jakarta.*
3. (빈 줄)
4. org.springframework.*
5. org.mybatis.*
6. (빈 줄)
7. com.plantynet.*
8. (빈 줄)
9. 서드파티 (io.jsonwebtoken.*, com.azure.*, com.google.firebase.* 등)
10. lombok.*
11. (빈 줄)
12. static import

16. Application 엔트리포인트

```
@SpringBootApplication(exclude = UserDetailsServiceImpl.class)
@ComponentScan("com.plantynet.gardenapi.dao")
@EnableAsync
@EnableTransactionManagement
public class GardenapiApplication { ... }
```

17. application.yml 주요 설정

항목	값
Server Context Path	/api
Server Port (local)	8000
MyBatis type-aliases-package	com.plantynet.gardenapi.entity
MyBatis mapper-locations	classpath:sql/**/*.xml
map-underscore-to-camel-case	true
auto-mapping-behavior	full
Async Thread Pool	prefix: gardenTask-, core: 4, max: 40, queue: 160

18. UserRole enum

enum 상수	key	설명
NON_SUBSCRIBER	RS001	일반 사용자 (회원가입 완료)
INDIVIDUAL	RS002	독립자
FAMILY_MEMBER	RS003	패밀리 구성원
FAMILY_MANGER	RS004	패밀리 관리자
ADMIN	AD001	운영 담당자
SUPER_ADMIN	AD002	슈퍼 관리자

JWT `role` 클레임에는 `ROLE_` 접두사가 붙는다 (예: `ROLE_RS003`). 서비스 코드에서 키 비교 시 `ROLE_` 를 제거한 값 (`RS003`)으로 비교한다.

19. 기술 스택

계층	기술
Framework	Spring Boot 3.5.14
Data Access	MyBatis 3.0.5 + MariaDB
Cache	Spring Data Redis
Document DB	Spring Data MongoDB
Message Queue	Spring Kafka
Security	Spring Security 6 + JWT (JJWT 0.13.0)
HTTP Client	Apache HttpClient5
Cloud Storage	Azure Blob Storage 12.34.0 + Azure Identity 1.18.3
Push Notification	Firebase Admin 9.9.0
XSS 방어	Lucy XSS Filter + 커스텀 Jackson 모듈
API 문서	SpringDoc OpenAPI 2.8.17 (<code>@Profile("local","dev")</code> 만 활성화)
ID 생성	TSID (tsid-creator 5.2.6)
Object Mapping	MapStruct 1.6.3
코드 생성	Lombok
분산 추적	Micrometer Tracing (OTEL bridge)

20. 코드 생성 시 자기 검토 체크리스트

- [] 여는 중괄호가 새 줄에 있는가? (Allman style)
- [] else / catch / finally도 닫는 `}` 다음 줄에 있는가?
- [] 들여쓰기에 탭을 사용했는가?
- [] `@Autowired` 필드 주입을 쓰지 않았는가?
- [] `@Data`, `val`, `var` 등 금지 어노테이션을 쓰지 않았는가? (`@AllArgsConstructor` 는 entity/, dto/response/ 패키지에서만 허용)
- [] `@Transactional` 을 서비스에 직접 붙이지 않았는가?
- [] 컨트롤러가 `BaseController` 를 상속하고 `resSuccess()` / `resCreated()` 를 사용하는가?
- [] 예외를 던질 때 `DomainException` 하위 클래스를 사용하는가?
- [] `DomainException` 생성자가 `super(ApiMessage.XXX)` 형태인가? (`HttpStatus` 전달 금지)
- [] Request DTO suffix가 `Req`, Response DTO suffix가 `Resp` 인가?
- [] Entity `boolean` 필드가 `isXxx` 가 아닌 `xxx` 형태인가?
- [] DAO 단일 결과가 `Optional<T>` 로 반환되는가?
- [] 레이어별 상세 규칙을 `references/` 파일에서 확인했는가?